

Ex1. Perform an Experiment for port scanning with nmap

Aim of the Experiment:

-To understand what a **network port** is, how to use **Nmap** to scan for open ports on a machine, and to distinguish between **ethical** and **unethical** uses of port scanning with real-world examples.

What is a Port?

Imagine a computer as a Hotel:

- A **hotel** has many **rooms**.
- Each **room** serves a different purpose (e.g., dining, sleeping, storage).
- A **computer** is like a **hotel**, and **ports** are its **rooms**.

Definition:

-A **port** is a **communication endpoint** on a computer where data is received and sent. Each port is associated with a specific service or application.

Port Numbers:

- Ports are numbered from **0 to 65535**.
- Common Port Numbers:

Port Number	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Secure remote login
23	Telnet	Unsecure remote login
25	SMTP	Sending email
53	DNS	Resolving domain names
80	HTTP	Web browsing
443	HTTPS	Secure web browsing
3306	MySQL	Database

Example:

- When you open <https://www.google.com>, your browser connects to **port 443** on Google's servers because it's using **HTTPS**.

What is Port Scanning?

-Port scanning is like checking which **doors (ports)** are open on a building (computer) to see which services are active. It helps in:

- Identifying vulnerabilities
- Auditing networks
- Ethical hacking

What is Nmap?

- **Nmap = Network Mapper**
- It's a powerful command-line tool used to:
 - Discover live hosts on a network
 - Identify open ports and services
 - Detect the operating system
 - Perform security audits

Tools Required:

Tool	Description
Nmap	Port scanner
Target Machine	Localhost / VM (Ubuntu, Windows, etc.)
OS	Kali Linux / Ubuntu / Windows
Network Setup	LAN or Virtual Network (do not scan public IPs without permission)

Types of Port States (Nmap output):

Port State	Meaning
Open	Service is listening (can connect)
Closed	No service is listening (but port exists)
Filtered	Port blocked by firewall
Unfiltered	Port accessible, but no info
Open	Filtered
Closed	Filtered

Nmap Command Reference Guide with Detailed Explanations

1. Basic Scanning Commands

Command	Description	Example	Detailed Explanation
nmap <IP>	Basic scan	nmap 192.168.1.1	Performs a default scan (TCP SYN on 1000 common ports) on the target IP. Useful for identifying open ports on a host.
nmap <domain>	Scan domain	nmap google.com	Resolves the domain to its IP address and scans it for open ports. Useful for scanning websites or remote servers.
nmap <subnet>/24	Subnet scan	nmap 192.168.1.0/24	Scans all 256 hosts in the subnet for live hosts and open ports. Excellent for discovering devices in a local network.

2. Port Scanning Options

Command	Description	Example	Detailed Explanation
-p <port>	Scan specific port	<code>nmap -p 22 192.168.1.1</code>	Checks if a specific port (e.g., SSH on port 22) is open on the target. Good for auditing known services.
-p-	Scan all ports	<code>nmap -p- 192.168.1.1</code>	Scans all 65535 TCP ports on the target. Useful for thorough investigations.
-F	Fast scan	<code>nmap -F 192.168.1.1</code>	Scans only the top 100 most common ports. Fast but may miss uncommon services.
-r	Ordered scan	<code>nmap -r 192.168.1.1</code>	Disables randomization of port scan order. Useful when predictable behavior is required.
--top-ports <n>	Scan top N ports	<code>nmap --top-ports 20 192.168.1.1</code>	Scans the top N ports most commonly used across the Internet. Faster than a full scan.

3. Scan Techniques

Command	Technique	Example	Detailed Explanation
-sS	SYN scan (Stealth Scan)	<code>nmap -sS 192.168.1.1</code>	Sends SYN packets and waits for a response. If a SYN-ACK is received, the port is open. If RST is received, the port is closed. It doesn't complete the handshake, making it stealthy and hard to detect. Requires root/admin privileges.
-sT	TCP Connect Scan	<code>nmap -sT 192.168.1.1</code>	Uses the OS's network API to complete a full TCP handshake. It's noisier than SYN scan but doesn't require root.

			Useful when user has limited privileges.
-sU	UDP Scan	<code>nmap -sU 192.168.1.1</code>	Sends empty UDP packets to specified ports. If ICMP Port Unreachable is returned, port is closed. If no response, port may be open or filtered. Slower than TCP due to lack of response feedback. Ideal for scanning services like DNS, SNMP, and TFTP.
-sN	Null Scan	<code>nmap -sN 192.168.1.1</code>	Sends packets with no TCP flags. Relies on target's TCP stack to identify open ports. Some systems respond with RST for closed ports and nothing for open ones. Effective against basic firewall filtering.
-sX	Xmas Scan	<code>nmap -sX 192.168.1.1</code>	Sends packets with FIN, URG, and PSH flags set, resembling a Christmas tree (all lights on). Useful for bypassing certain firewall rules and detecting open ports based on how target responds.
-sF	FIN Scan	<code>nmap -sF 192.168.1.1</code>	Sends TCP packets with only the FIN flag set. Open ports often ignore the packet while closed ports respond with RST. Used to evade firewalls that block SYN packets.

4. Service & Version Detection

Command	Description	Example	Detailed Explanation
-sV	Service version detection	<code>nmap -sV 192.168.1.1</code>	Queries open ports to detect service versions (e.g., Apache 2.4.29). Important for vulnerability assessment.
--version-intensity <0-9>	Adjust detection depth	<code>nmap --version-intensity 5</code>	Controls how aggressively Nmap probes ports. Lower values are faster but less accurate.

5. OS Detection

Command	Description	Example	Detailed Explanation
-O	Operating System detection	<code>nmap -O 192.168.1.1</code>	Uses TCP/IP stack fingerprinting to identify the OS of the target host.
--osscan-guess	Aggressive OS guessing	<code>nmap -O --osscan-guess</code>	Attempts to guess the OS even when certainty is low. Useful in mixed environments.

6. Aggressive Scan

Command	Description	Example	Detailed Explanation
-A	Aggressive scan	<code>nmap -A 192.168.1.1</code>	Combines OS detection, version detection, script scanning, and traceroute. Ideal for full reconnaissance.

7. Script Scanning (NSE)

Command	Description	Example	Detailed Explanation
-sC	Default scripts	<code>nmap -sC 192.168.1.1</code>	Runs a set of scripts for common vulnerabilities and info gathering.
--script <script>	Specific script	<code>nmap --script http-title</code>	Executes a single script, like retrieving the title of a web page.
--script vuln	Vulnerability scripts	<code>nmap --script vuln 192.168.1.1</code>	Runs multiple vulnerability detection scripts. Very useful for pentesting.

8. Timing & Performance

Command	Description	Example	Detailed Explanation
-T<0-5>	Timing template	<code>nmap -T4 192.168.1.1</code>	Controls scan speed. T0 = slowest/stealthy, T5 = fastest/loudest.
--min-rate <n>	Set min packet rate	<code>nmap --min-rate 1000</code>	Ensures Nmap sends at least N packets per second. Useful for fast scans.
--max-retries <n>	Retry limit	<code>nmap --max-retries 2</code>	Sets how many times Nmap retries a probe before giving up.

9. Output Options

Command	Description	Example	Detailed Explanation
-oN <file>	Normal output	<code>nmap -oN scan.txt</code>	Human-readable output written to a file.
-oX <file>	XML output	<code>nmap -oX scan.xml</code>	XML format for parsing and automation.
-oG <file>	Grepable output	<code>nmap -oG scan.grep</code>	Easy to filter using grep/awk. Useful for scripting.
-oA <prefix>	All formats	<code>nmap -oA myscan</code>	Creates .nmap, .xml, and .grep files with given prefix.

10. Host Discovery

Command	Description	Example	Detailed Explanation
-sn	Ping scan	<code>nmap -sn 192.168.1.0/24</code>	Finds which hosts are up without scanning ports.
-Pn	No ping	<code>nmap -Pn 192.168.1.1</code>	Skips host discovery; assumes target is online.
-PS	TCP SYN ping	<code>nmap -PS80,443 192.168.1.1</code>	Sends SYN packets to specified ports to discover live hosts.

-PA	TCP ACK ping	<code>nmap -PA80,443</code>	Sends ACK packets to detect hosts that allow incoming traffic.
-PU	UDP ping	<code>nmap -PU53</code>	Sends UDP packets to test if host responds.

11. Firewall/IDS Evasion

Command	Description	Example	Detailed Explanation
-f	Fragment packets	<code>nmap -f 192.168.1.1</code>	Breaks up packets to evade simple firewall rules.
--source-port <port>	Set source port	<code>nmap --source-port 53</code>	Tricks firewalls into thinking traffic is DNS or another trusted service.
-D RND:10	Decoy scan	<code>nmap -D RND:10 192.168.1.1</code>	Sends fake scans from random IPs to mask real source.
--data-length <n>	Pad packets	<code>nmap --data-length 50</code>	Adds junk data to packets to obscure signature.

Full Command Example:

`nmap -sS -p 80 --source-port 53 192.168.1.1`

Explanation of Each Part:

Part	Meaning
nmap	Start the Nmap tool
-sS	Use TCP SYN scan (stealthy)
-p 80	Scan port 80 (HTTP)
--source-port 53	Set source port to 53 (DNS)
192.168.1.1	Target IP address

Use Case:

Some firewalls **trust traffic from port 53** (used by DNS), so attackers or pentesters may use this trick to **bypass basic filters** or **evade intrusion detection systems (IDS)**.

Important Notes:

- Only works if you run the scan with **root privileges** (use sudo)

Example:

```
sudo nmap -sS -p 80 --source-port 53 192.168.1.1
```

- This technique is for **ethical testing only** and should be used in **authorized environments**.

Full Command Example:

1. **--data-length <n>**

Adds extra random data to each packet to obfuscate scans.

```
sudo nmap -sS -p 80 --data-length 50 192.168.1.1
```

Use Case:

This adds **50 bytes of random data** to each packet to **evade signature-based IDS/IPS** systems by making the packets look different from standard Nmap scans.

2. **-f (Fragment Packets)**

Splits packets into tiny pieces (fragments) to bypass simple firewalls or intrusion detection systems.

```
sudo nmap -sS -p 80 -f 192.168.1.1
```

Use Case:

Some **packet filters can't reassemble fragmented packets**, so this may help bypass those filters. It's a stealthy technique used in penetration testing.

3. **-D (Decoy Scanning)**

Spoofs multiple IP addresses to confuse the target about your real IP.

```
sudo nmap -sS -p 80 -D RND:10 192.168.1.1
```

Use Case:

This tells Nmap to generate **10 random decoy IPs**, so the target sees many incoming scans and **can't easily tell which is real**.

Example: Useful for **bypassing logging or honeypots** during red team operations.

Combine Multiple Evasion Techniques:

```
sudo nmap -sS -p 80 --source-port 53 --data-length 100 -f -D RND:5 192.168.1.1
```

This:

- Uses **DNS port 53** as the source port
- Adds **100 bytes of random data**
- **Fragments** packets
- Spoofs **5 decoys**
- Targets **port 80** (HTTP)

⚠ **Ethical Reminder:**

These techniques should only be used on systems you have **permission** to test, like:

- Internal networks
- Training environments (e.g., Hack The Box, TryHackMe)
- Client-approved penetration tests

12. Scanning Multiple Targets

Command	Description	Example	Detailed Explanation
nmap 192.168.1.1-10	IP range	nmap 192.168.1.1-254	Scans a range of IP addresses in sequence.
nmap -iL list.txt	Input from file	nmap -iL ips.txt	Loads a list of target IPs from a text file.
nmap -iR 5	Random IPs	nmap -iR 5	Scans randomly chosen IP addresses. Useful for testing.
nmap --exclude <ip>	Exclude hosts	nmap --exclude 192.168.1.5	Skips scanning specific IPs.

13. Real-World Use Cases

Use Case	Example Command	Explanation
Network Inventory	<code>nmap -sn 192.168.0.0/24</code>	Lists all live hosts in the subnet.
Find Open Web Servers	<code>nmap -p 80,443 -sV 192.168.1.0/24</code>	Detects live websites and service versions.
Detect Vulnerabilities	<code>nmap --script vuln 192.168.1.1</code>	Checks for known CVEs on open ports.
Audit SSH Security	<code>nmap --script ssh* -p 22 192.168.1.1</code>	Runs all SSH-related scripts.
Check Database Exposure	<code>nmap -p 3306 --script mysql* 192.168.1.1</code>	Detects exposed MySQL services.
Identify IoT Devices	<code>nmap -O -sV 192.168.1.1</code>	Matches services and OS to known IoT fingerprints.
Firewall Evasion (lab only)	<code>nmap -f</code> or <code>nmap -D RND:10</code>	Tests firewall bypass methods.

14. Advanced Script Scanning

Command	Description	Example	Use Case
<code>--script-help <script></code>	Show script description	<code>nmap --script-help http-title</code>	Get help for a specific NSE script
<code>--script-args <args></code>	Pass arguments to NSE script	<code>nmap --script http-brute --script-args userdb=users.txt,passdb=pass.txt</code>	Run brute-force attack with custom credentials
<code>--script-args-file <file></code>	Load NSE args from a file	<code>nmap --script-args-file args.txt</code>	Reuse custom script inputs from file

15. Authentication and Safe Mode

Command	Description	Example	Use Case
--script-trace	Show all packets sent by scripts	<code>nmap --script-trace -sC 192.168.1.1</code>	Debug how scripts interact with hosts
--privileged	Force privileged scan	<code>nmap --privileged -sS</code>	Force use of raw sockets in restricted shells
--unprivileged	Force unprivileged scan mode	<code>nmap --unprivileged -sT</code>	Simulate behavior of a non-root user

16. Performance and Parallelism

Command	Description	Example	Use Case
--min-parallelism <num>	Set minimum parallel probes	<code>nmap --min-parallelism 10 192.168.1.1</code>	Speed up scans on fast networks
--max-parallelism <num>	Set maximum parallel probes	<code>nmap --max-parallelism 100 192.168.1.1</code>	Avoid overwhelming network/host
--scan-delay <time>	Set delay between probes	<code>nmap --scan-delay 1s 192.168.1.1</code>	Evade rate-limiting firewalls

17. IPv6 Scanning

Command	Description	Example	Use Case
<code>nmap -6 <IPv6></code>	Scan IPv6 address	<code>nmap -6 2001:db8::1</code>	Scan modern networks using IPv6

18. NSE Categories (Script Groups)

Category	Description	Example
auth	Authentication checks	<code>nmap --script auth 192.168.1.1</code>
vuln	Known vulnerabilities	<code>nmap --script vuln 192.168.1.1</code>
default	Default scripts	<code>nmap -sC 192.168.1.1</code>
discovery	Network and host discovery	<code>nmap --script discovery 192.168.1.1</code>
malware	Malware scanning	<code>nmap --script malware 192.168.1.1</code>
brute	Brute-force attacks	<code>nmap --script brute 192.168.1.1</code>
exploit	Exploit known vulnerabilities	<code>nmap --script exploit 192.168.1.1</code>

To view available scripts in a category:

```
nmap --script-help <category>
```

19. Save and Resume Scans

Command	Description	Example	Use Case
<code>--resume <file></code>	Resume an interrupted scan	<code>nmap --resume myscan.log</code>	Continue a long scan without restarting
<code>--datadir <path></code>	Use custom Nmap data directory	<code>nmap --datadir /opt/nmap</code>	Load scripts/fingerprints from custom path

20. NSE Update and Integration

Command	Description	Example	Use Case
--script-updatedb	Update local script DB	<code>sudo nmap --script-updatedb</code>	After adding or modifying scripts
-oX	XML output	<code>nmap -oX result.xml</code>	Use results in other tools like Metasploit

Extra Tips

- View help: `nmap --help`
- Read the manual: `man nmap`
- List scripts: `ls /usr/share/nmap/scripts/`
- Practice on safe targets like scanme.nmap.org

Note: Nmap now supports over **1000+** NSE scripts, powerful evasion methods, and performance options for both beginners and experts.

21. Advanced Output Options

Command	Description	Example	Use Case
--append-output	Append to output file	<code>nmap -oN scan.txt --append-output</code>	Continue logging into the same file across scans
--webxml	Create XML for web viewer	<code>nmap -oX scan.xml --webxml</code>	Format designed for web browsers
--reason	Show port state reasons	<code>nmap --reason 192.168.1.1</code>	Explains why a port is open/closed
--open	Only show open ports	<code>nmap --open 192.168.1.1</code>	Cleaner results by hiding closed/filtered ports

--iflist	List network interfaces	<code>nmap --iflist</code>	Shows available interfaces/IPs (debugging)
--stylesheet <file>	Attach XSLT to XML	<code>nmap -oX scan.xml --stylesheet style.xsl</code>	Beautify XML output in browsers

22. Advanced Host Discovery

Command	Description	Example	Use Case
--traceroute	Trace route to target	<code>nmap --traceroute 192.168.1.1</code>	Identify intermediate hops
--dns-servers <servers>	Use specific DNS	<code>nmap --dns-servers 1.1.1.1,8.8.8.8</code>	Override system DNS
--system-dns	Use system-configured DNS	<code>nmap --system-dns</code>	Fall back to system DNS if others fail
--resolve-all	Resolve all hostnames	<code>nmap --resolve-all google.com</code>	Find all DNS IPs for a domain

23. Packet Crafting & Customization

Command	Description	Example	Use Case
--ttl <value>	Set packet TTL	<code>nmap --ttl 10 192.168.1.1</code>	Test hop filters or TTL behavior
--badsum	Send bad checksum	<code>nmap --badsum 192.168.1.1</code>	IDS/IPS testing, packets should be dropped
--mtu <size>	Fragment packets	<code>nmap --mtu 24 192.168.1.1</code>	Obfuscate scan to evade firewalls

24. Target Specification Enhancements

Command	Description	Example	Use Case
--exclude-file <file>	Exclude targets via file	<code>nmap -iL targets.txt --exclude-file skip.txt</code>	Fine control over target lists
-sL <range>	List targets only	<code>nmap -sL 192.168.1.0/24</code>	Preview scan scope (no packets sent)
-iL <file>	Load IPs from file	<code>nmap -iL myips.txt</code>	Batch scanning from saved list

25. Debugging and Troubleshooting

Command	Description	Example	Use Case
-d, -d2 to -d9	Debug output	<code>nmap -d3 192.168.1.1</code>	See detailed scan logic and flow
--packet-trace	Trace packets	<code>nmap --packet-trace</code>	Show each packet sent/received
--reason	Show port reasons	<code>nmap --reason</code>	Know why port status is assigned

26. Performance & Reliability Tuning

Command	Description	Example	Use Case
--host-timeout <time>	Max time per host	<code>nmap --host-timeout 5m</code>	Stop slow scans from stalling full scan
--max-scan-delay <time>	Cap delay between probes	<code>nmap --max-scan-delay 100ms</code>	Improve efficiency
--max-rtt-timeout <time>	Max round-trip wait	<code>nmap --max-rtt-timeout 1s</code>	Ensure timely port response cutoff
--initial-rtt-timeout <time>	Set initial RTT	<code>nmap --initial-rtt-timeout 200ms</code>	Tweak scan speed in fast/slow networks

27. Nmap GUI (Zenmap)

Command	Description	Example	Use Case
zenmap	Launch GUI interface	zenmap	Graphical version of Nmap (if installed)
Save profiles	Create custom scan profiles	—	For repeated scans (e.g., daily web audit)

Zenmap is a graphical front-end for Nmap with features like scan history, topology maps, and result comparison. Use it if you prefer visual tools.

28. Ndiff (Compare Nmap Outputs)

Command	Description	Example	Use Case
ndiff file1.xml file2.xml	Compare two Nmap scans	ndiff before.xml after.xml	Spot changes between scans (added/removed hosts/services)

29. NSE Performance Flags (Advanced)

Command	Description	Example	Use Case
--script-timeout <time>	Set NSE script timeout	--script-timeout 20s	Avoid long-running scripts in large scans
--script-trace	Show NSE script packet traces	--script-trace	For debugging or transparency
--script-updatedb	Update NSE script DB	sudo nmap --script-updatedb	After adding new custom scripts

Full Command Example:

```
nmap --script vuln --script-timeout 20s 192.168.1.1
```

Explanation of Each Part:

Part	Meaning
nmap	Starts the Nmap tool

--script vuln	Runs vulnerability detection scripts
--script-timeout 20s	Limits each script to run for max 20 seconds
192.168.1.1	Target IP address

Use Case:

You're scanning a server for known vulnerabilities, but you don't want the scan to hang because of slow scripts. So you **limit each script to 20 seconds** — anything slower gets skipped.

Tips:

- You can use 10s, 30s, 1m, etc.
- Combine with other options like -sV or -A if needed.

30. Custom NSE Script Usage

If you write your own NSE script (e.g., myscan.nse), use:

```
nmap --script ./myscan.nse <target>
```

Write custom scripts using Lua. This is powerful for building:

- Malware detectors
- Custom brute-force checks
- Exploit attempts for known vulnerabilities

31. Location and Resource Paths

Command	Description	Example
nmap --datadir <path>	Custom data path	nmap --datadir /opt/nmap
nmap --script-updatedb	Update script cache	sudo nmap --script-updatedb
ls /usr/share/nmap/scripts/	View installed scripts	For script discovery

32. Combine with Other Tools

- **With Metasploit:** Use -oX XML output in Metasploit (db_import).
- **With Wireshark:** Use --packet-trace or tcpdump alongside Nmap.
- **With cron jobs:** Automate regular scans with logging.

Output Options

Nmap can save its results in different formats so you don't lose them.

Option	Use	Example	Description
-oN scan.txt	Normal	<code>nmap -oN scan.txt 192.168.1.1</code>	Saves output as text
-oX scan.xml	XML	<code>nmap -oX scan.xml</code>	Saves in XML (for automation)
-oG scan.grep	Grepable	<code>nmap -oG scan.grep</code>	Easy to filter using grep
-oA allformats	All formats	<code>nmap -oA myscan</code>	Saves .nmap, .xml, and .grep

Observations:

Command	Purpose	Your Observation
---------	---------	------------------

Real-Time Scenarios (Examples):

Legal Scenario:

-A security team scans their company's internal servers to find open ports for maintenance — **with permission**.

Illegal Scenario:

-A hacker scans a bank's website without permission and uses open ports to exploit a server — **without permission**, violates the **IT Act 66C/66D** in India.

Legal vs Illegal Use of Port Scanning:

Action	Legal	Illegal	Example
Scan your own PC/server	✓	✗	Test open services on localhost
Scan a public IP without asking	✗	✓	Scanning Netflix.com
Scan a friend's PC with permission	✓	✗	Lab practice
Scan government servers	✗	✓	Against the law

Additional Tips for Better Understanding:

- Use nmap localhost to safely test on your own machine.
- Use nmap -F for a **fast scan** (top 100 ports).
- Use nmap -O to detect the **OS** of the target (in aggressive mode).
- Always **log your scans** with -o filename.txt.

Precautions and Ethics:

- **Never** scan unknown IPs without written permission.
- Respect **cyber laws** and **digital ethics**.
- Use tools only in labs, sandboxes, or for **certified penetration testing**.
- Educate others on legal/illegal boundaries of scanning.

What Can I Do If I Know Port Scanning Well?

- - Learn vulnerability assessment and penetration testing (VAPT)
- - Practice ethical hacking using platforms like Hack The Box, TryHackMe
- - Get certified (e.g., CEH, CompTIA Security+, OSCP)
- - Learn advanced tools like Wireshark, Nessus, Metasploit
- - Join bug bounty programs (HackerOne, Bugcrowd)

Result:

This experiment taught:

- The **concept of ports and services**
- **Nmap** usage at various levels (basic to advanced)
- The importance of **ethical hacking**
- Differences between **safe/unsafe, legal/illegal** scanning

Port scanning is a powerful tool — but with great power comes great responsibility!

EXP-2: Experiment for Set up a honey pot and monitor the honey pot on the network

What is a Honeypot?

A **honeypot** is a **fake system or service** designed to look vulnerable, so that **attackers are tricked into interacting** with it.

We can then **monitor, analyze, and log their behavior** — like open ports they probe, malware they upload, etc.

Step-by-Step: Set Up and Monitor a Honeypot on the Network

Step 1: Choose the Right Environment

You can run your honeypot on:

- A **Virtual Machine (VM)** (best for isolation)
- A **Docker container**
- A **physical device** (like Raspberry Pi)
- In the **cloud** (with caution — it can get attacked fast)

For beginners, a **VM running Kali Linux or Ubuntu** is recommended.

Step 2: Install a Honeypot Tool

There are many open-source honeypots. Here are 3 good options:

Honeypot Name	Purpose	Easy to Use?
Cowrie	SSH & Telnet trap	★★★★
Dionaea	Malware collection	★★★
Kippo	Older SSH honeypot	★★
Honeyd	Simulates entire fake networks	★★★

Note: We'll use **Cowrie** for this example — it's a powerful SSH honeypot.

Step 3: Install Cowrie (SSH Honeypot)

Step-by-Step on Ubuntu/Kali Linux:

Step 1: Install dependencies

```
sudo apt update
```

```
sudo apt install git python3-venv python3-dev libssl-dev libffi-dev build-essential
```

Step 2: Clone the Cowrie repository

```
git clone https://github.com/cowrie/cowrie.git
```

```
cd cowrie
```

Step 3: Create a virtual environment

```
python3 -m venv cowrie-env
```

```
source cowrie-env/bin/activate
```

Step 4: Install Python requirements

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

Step 5: Copy default config

```
cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

Step 6: Start honeypot

```
bin/cowrie start
```

Now Cowrie will listen on a **fake SSH port (usually 2222)**.

Step 4: Monitor the Honeypot

Cowrie logs **everything attackers do**, like:

- Login attempts
- Commands typed
- Files uploaded

You can see logs here:

```
tail -f var/log/cowrie/cowrie.log
```

Example log:

2025-06-27T10:35Z login attempt with username: root and password: 123456

You can also record session replays:

```
bin/playlog log/tty/...
```

Step 5: Redirect Real Port 22 to Cowrie Port 2222

You can use iptables to forward real SSH traffic:

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-ports 2222
```

So when attackers try to connect to port 22, it goes to Cowrie.

⚠ Important: Do not expose this on your production system unless isolated properly.

Watch in Real-Time (Optional Monitoring Tools)

Use tools to visualize attacks:

- **Kibana + Elasticsearch:** Great for dashboards
- **Grafana + Loki:** Logs in real-time
- **DSshield Honeypot project:** Share data with global researchers

Summary of Commands

Setup

```
git clone https://github.com/cowrie/cowrie.git
cd cowrie
python3 -m venv cowrie-env
source cowrie-env/bin/activate
pip install -r requirements.txt
cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

Start honeypot

```
bin/cowrie start
```

View logs

```
tail -f var/log/cowrie/cowrie.log
```

Optional: redirect SSH port

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-ports 2222
```

Use Case: What You Learn from a Honeypot?

- Common usernames/passwords attackers try
- Typical scripts or malware they upload
- Tools attackers use (e.g., wget, curl, netcat)
- Their IP addresses and behavior patterns

Note: Security Tip

- A honeypot should never be run on a critical production system.
- Use it in isolated networks or VMs only. Honey pots invite attacks, so safety first!

EXP-3: Install Jscript/Cryptool tool (or any other equivalent) and demonstrate Asymmetric, Symmetric crypto algorithm, Hash and Digital/PKI signatures.

Tool Used: OpenSSL (CLI)

Equivalent Tools: CryptoTool (GUI), GnuPG (CLI), VeraCrypt (GUI), Python cryptography library.

1. Symmetric Encryption & Decryption (AES-256-CBC)

Concept: Same key is used to encrypt and decrypt.

Algorithm: AES-256-CBC (Advanced Encryption Standard, 256-bit key, Cipher Block Chaining mode).

Step 1: Create a file to encrypt

```
echo "This is my secret message." > secret.txt
```

Step 2: Encrypt the file

```
openssl enc -aes-256-cbc -salt -in secret.txt -out secret.enc
```

- **-aes-256-cbc** → Algorithm
 - **-salt** → Adds randomness to encryption
 - **-in** → Input file
 - **-out** → Output encrypted file
- You'll be asked for a password — **remember it.**

Step 3: Check if file is encrypted

```
cat secret.enc
```

- If you see **gibberish characters**, encryption worked.
- If you see readable text, something went wrong.

Step 4: Decrypt the file

```
openssl enc -aes-256-cbc -d -in secret.enc -out decrypted.txt
```

- **-d** → Decrypt mode

Step 5: Verify decryption

```
cat decrypted.txt
```

- Should display:
This is my secret message.

2. Asymmetric Encryption & Decryption (RSA)

Concept: Public key encrypts, private key decrypts.

Algorithm: RSA 2048-bit.

Step 1: Generate RSA key pair

```
openssl genrsa -out private.pem 2048  
openssl rsa -in private.pem -pubout -out public.pem
```

Step 2: Encrypt using the public key

```
openssl rsautl -encrypt -inkey public.pem -pubin -in secret.txt -out secret_rsa.enc
```

Step 3: Check if encrypted

```
cat secret_rsa.enc
```

- Should be binary/gibberish.

Step 4: Decrypt using the private key

```
openssl rsautl -decrypt -inkey private.pem -in secret_rsa.enc -out rsa_decrypted.txt
```

Step 5: Verify decryption

```
cat rsa_decrypted.txt
```

- Should match original message.

3. Hashing (SHA-256)

Concept: One-way function — can't be reversed.

Step 1: Create hash

```
openssl dgst -sha256 secret.txt
```

Example output:

SHA256(secret.txt)= 3a7bd3e2360a3d...

Step 2: Test change detection

```
echo "new line" >> secret.txt  
openssl dgst -sha256 secret.txt
```

- Hash will be **completely different**.

4. Digital Signatures (PKI)

Concept: Proves authenticity and integrity.

Step 1: Sign a file

```
openssl dgst -sha256 -sign private.pem -out signature.bin secret.txt
```

Step 2: Verify the signature

```
openssl dgst -sha256 -verify public.pem -signature signature.bin secret.txt
```

OutPut:

Verified OK

If file is changed:

Verification Failure

EXP-3: Generate minimum 10 passwords of length 12 characters using openssl command

Generating Random Passwords with OpenSSL

Concept: Using OpenSSL's rand command to create cryptographically secure random strings.

Command:

```
for i in {1..10}; do  
    openssl rand -base64 12  
done
```

Explanation:

- for i in {1..10} → Loops 10 times.
- openssl rand → Generates random bytes.
- -base64 → Encodes in Base64 (makes it keyboard-friendly).
- 12 → Number of random bytes (Base64 output will be longer than 12 characters).

Sample Output:

```
gqK7F2cYvVZ4rV+2  
zO7S9pHnA4q2K7yG  
N6+Z3K0oVhZrC5lY  
2u8+qEoV1zCk6iGb  
qL5vZ2CkA0XhQ8yP  
rCzG5lN8wO0hK7vQ  
W2+oQpV3lK6jY7aG  
F3xK9uQvP1wH4lZr  
gQ9yV0uP8hZrB7kT  
lZpK2hV6qR8yJ4nC
```

If you need exactly 12 visible characters (no extra Base64 padding):

You can trim the output using cut:

```
for i in {1..10}; do
  openssl rand -base64 12 | cut -c1-12
done
```

Sample Output:

```
gqK7F2cYvVZ4
zO7S9pHnA4q2
N6+Z3K0oVhZr
2u8+qEoV1zCk
qL5vZ2CkA0Xh
rCzG5lN8wO0h
W2+oQpV3lK6j
F3xK9uQvP1wH
gQ9yV0uP8hZr
lZpK2hV6qR8y
```

That one is a **single command** — it just looks long because it's written across multiple lines for readability.

In reality, you can paste it **exactly like this** in your terminal:

```
for i in {1..10}; do openssl rand -base64 12 | cut -c1-12; done
```

This is **not** a separate script file — it's just a **one-liner loop command** in bash that:

1. Loops from 1 to 10 (for i in {1..10})
2. Runs openssl rand -base64 12 (generate random bytes, then encode in base64)
3. Cuts the result to 12 characters (cut -c1-12)
4. Prints each password

EXP-5: Perform practical approach to implement Foot printing - Gathering target information using Dmitry-Dmagic, UA tester

1. Objective

Perform **foot printing** — the process of gathering publicly available information about a target before launching a penetration test — using:

- **Dmitry / Deepmagic** (dmitry or dmitry -dmagic)
- **UA Tester** (changing your browser or tool's user agent)

2. Tools Needed

Tool	Purpose	Installation
Dmitry	All-in-one information gathering tool	sudo apt install dmitry
Curl / Browser UA Switcher	Change & test User-Agent	Curl comes pre-installed on most Linux

3. Understanding Dmitry

Dmitry can perform:

- **Whois lookups**
- **Netcraft lookups**
- **Port scans**
- **Subnet search**
- **Email address search**
- **Reverse lookups**

4. Basic Dmitry Command Structure

dmitry [options] target

Options:

Option	Function
-i	Perform whois lookup on IP address
-w	Perform whois lookup on domain
-n	Perform Netcraft.com search
-s	Perform subdomain search
-e	Search for email addresses
-p	Perform TCP port scan
-f	Perform TCP port scan with banner grabbing

5. Practical Steps – Dmitry

Step 1 – Install Dmitry

```
sudo apt update  
sudo apt install dmitry
```

Step 2 – Basic Info Gathering

```
dmitry -winse target.com
```

Example: `dmitry -winse google.com`

```
dmitry -winse swarnandhra.ac.in
```

What this does:

- -w → Whois on domain
- -i → Whois on IP
- -n → Netcraft info
- -s → Subdomain search
- -e → Email search

Step 3 – Port Scan & Banner Grab

```
dmitry -p -f target.com
```

- -p → Port scan
- -f → Banner grabbing (reveals service versions)

Step 4 – Save Results

```
dmitry -winsep target.com -o result.txt
```

- -o file → Save output to file

6. UA Testing (User Agent Testing)

Why?

Some sites show different content depending on your User-Agent (browser type).

Example with curl:

```
curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64)" http://target.com
```

- -A sets the User-Agent to mimic a Windows browser.

Example – Pretend to be Googlebot:

```
curl -A "Googlebot/2.1 (+http://www.google.com/bot.html)" http://target.com
```

7. Sample Lab Exercise

1. Open terminal in **Kali Linux**.
2. Install Dmitry:
`sudo apt install dmitry`
3. Run:
`dmitry -winse example.com`
4. Save to file:
`dmitry -winsep example.com -o example_results.txt`
5. Check file:
`cat example_results.txt`
6. Test UA change:
`curl -A "Mozilla/5.0 (Linux; Android 10)" http://example.com`

8. Expected Output

- Whois data
- Netcraft hosting info
- Subdomains found
- Email addresses from public sources
- Open ports list with banners
- Different website responses for different User-Agents

Footprinting & Tools Explanation

What is Footprinting?

- Footprinting means **collecting information** about a target computer system, network, or website **before** you try to find weaknesses or hack it.
- Think of it as doing your homework or detective work to know everything about your target from public sources.

Why Footprinting is Important?

- Helps to find:
 - What operating system the target uses (Windows, Linux, etc.)
 - What services (like web servers, email servers) are running
 - Open ports that can be accessed
 - Email addresses or domain info
- This info helps hackers or security testers plan their next move.

Key Tools & Terms

1. Dmitry (Deepmagic Information Gathering Tool)

- A simple **command-line tool** for collecting info on websites or IP addresses.
- It can do things like:
 - Get domain ownership info (Whois)
 - Find subdomains (like blog.example.com)
 - Search for email addresses linked to the domain
 - Scan for open network ports
 - Grab banners (details services send back when you connect)

Why use Dmitry?

It automates many info-gathering steps in one command.

2. Whois

- A public database showing **who owns a domain or IP**.
- Contains info like owner's name, contact details, registration date.

3. Netcraft

- A website/service that collects info about **web hosting and server technology**.
- Can tell which company hosts a website, server OS, SSL cert details.

4. Subdomain

- Like an **address inside a website's domain**.
- For example, in blog.example.com, blog is the subdomain of example.com.
- Subdomains can expose extra services or apps.

5. Email Search

- Footprinting tries to find email addresses associated with the target domain, useful for social engineering or phishing.

6. Port Scanning

- Computers communicate over **ports** — like doors or channels for different services (HTTP on port 80, SSH on port 22).
- Port scanning checks which “doors” are open on a machine.
- Open ports can reveal what services are running and might be vulnerable.

7. Banner Grabbing

- When you connect to an open port, some services send back a “banner” — a short text that tells you the service name and version.
- For example, an SSH server might send back OpenSSH 7.9p1.
- Knowing versions helps find vulnerabilities.

8. User-Agent (UA)

- When your browser talks to a website, it sends info called a **User-Agent** string, telling the site which browser and OS you use.
- Some websites show different content depending on the UA (for mobile browsers, Google bots, etc.).
- Changing UA lets you test how the site behaves for different devices or bots.

9. Curl

- A **command-line tool** to fetch webpages or communicate with servers.
- You can change the User-Agent with curl to mimic different browsers or bots.

How all these fit together?

- You use **Dmitry** to gather as much public info as possible about a target domain (who owns it, subdomains, emails, open ports).
- You use **port scanning and banner grabbing** to see which services the target runs and what versions.
- You test with different **User-Agents** using **curl** or browser extensions to see how the site reacts.
- All this info helps plan further tests or attacks.

Practical Footprinting Guide: Using Dmitry & User-Agent Testing

Step 1: Open Your Terminal

On Kali Linux, open Terminal by clicking on the terminal icon or pressing Ctrl+Alt+T

Step 2: Install Dmitry

Type this to install Dmitry if not installed yet:

```
sudo apt update  
sudo apt install dmitry -y
```

Step 3: Basic Dmitry Scan

Run this command to gather info about a target domain (replace example.com with your target):

```
dmitry -winse example.com
```

What does this do?

- -w → Whois info about the domain (owner, registrar)
- -i → Whois info about the IP address
- -n → Netcraft info about the domain's hosting
- -s → Search for subdomains
- -e → Search for email addresses linked to domain

Step 4: Port Scanning & Banner Grabbing

Scan open ports and grab banners:

```
dmitry -pf example.com
```

- -p → Scan TCP ports to find open “doors”
- -f → Try to grab banner info from services

Step 5: Save Output to File

You can save all results to a file to review later:

```
dmitry -winsep example.com -o footprint.txt
```

Then view it with:

```
cat footprint.txt
```

Step 6: User-Agent Testing Using Curl

Some websites behave differently for different browsers or devices. You can simulate this with curl by changing the User-Agent.

Example: Pretend you are using Chrome on Windows

```
curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64)" http://example.com
```

Example: Pretend you are Googlebot (Google's crawler)

```
curl -A "Googlebot/2.1 (+http://www.google.com/bot.html)" http://example.com
```

Step 7: Experiment

Try changing User-Agent strings and see if website content changes.

More Understanding Dmitry

Dmitry (Deepmagic Information Gathering Tool) is a command-line tool used for **information gathering (footprinting)** during penetration testing or ethical hacking. It automates basic reconnaissance tasks against a given target (domain or IP).

Dmitry Can Perform:

1. Whois Lookups

- Retrieves **registration details** of a domain or IP address.
- Shows **owner info, creation/expiry date, registrar, name servers** etc.
- Example:
- `dmitry -w google.com`

→ Finds who registered google.com and when it expires.

2. Netcraft Lookups

- Uses **Netcraft.com** service to gather information about websites.
- Can reveal:
 - Hosting provider
 - Operating system & web server
 - SSL certificate details
- Example:

- `dmitry -i google.com`

3. Port Scans

- Scans the target host for **open TCP ports**.
- Useful for finding services (e.g., HTTP on port 80, SSH on port 22).
- Example:
- `dmitry -p google.com`

4. Subnet Search

- Attempts to identify **other hosts in the same subnet** as the target.
- Helps find **related servers** or hidden infrastructure.
- Example:

- `dmitry -s google.com`

5. Email Address Search

- Searches for **email addresses** related to the target domain.
- Can scrape from web pages or databases.
- Example:
- `dmitry -e google.com`

6. Reverse Lookups

- Performs **reverse DNS lookups** on IP addresses.
- Converts IP → Hostname.
- Useful for discovering **subdomains/servers**.
- Example:
- `dmitry -r 142.251.220.14`

All-in-One Example

You can combine options together:

`dmitry -winse target.com`

- `-w` = Whois
- `-i` = Netcraft
- `-n` = Netcraft DNS
- `-s` = Subdomain search
- `-e` = Email search

This gives a **full report** on the target.

In short: **Dmitry = Swiss army knife of reconnaissance.**

“It doesn’t exploit vulnerabilities, but it gives you **the first layer of information** about your target.”

Dmitry Functions – Comparison Table

Function	Flag	What it Does	Example Command	Example Output (Simplified)
Whois Lookup	-w	Gets registration info of a domain or IP (owner, registrar, creation/expiry, name servers).	dmitry -w google.com	Registrar: MarkMonitor Created: 1997-09-15 Expires: 2028-09-14 Name Servers: ns1.google.com ...
Netcraft Lookup	-i	Uses Netcraft to get hosting info (OS, server, hosting provider, SSL).	dmitry -i google.com	Hosting: Google LLC Server: gws SSL: Google Internet Authority
Port Scan	-p	Scans for open TCP ports on the target.	dmitry -p google.com	Open ports: 80/tcp → HTTP 443/tcp → HTTPS
Subnet Search	-s	Looks for other hosts in the same subnet.	dmitry -s google.com	Found hosts: 142.251.220.10 → mail.google.com 142.251.220.14 → google.com
Email Search	-e	Searches web pages for email addresses linked to the domain.	dmitry -e google.com	Found: support@google.com, abuse@google.com
Reverse Lookup	-r	Converts IP → Hostname (reverse DNS).	dmitry -r 142.251.220.14	IP: 142.251.220.14 → Hostname: google.com

Sample OutPut:

```
(kali@kali)~$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.2 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [51.8 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [119 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [325 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [200 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [911 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11.3 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [27.2 kB]
Fetched 74.7 MB in 12s (6,476 kB/s)
2240 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
(kali㉿ kali)-[~]
$ sudo apt install dmitry
dmitry is already the newest version (1.3a-6).
dmitry set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2240
```

```
(kali㉿ kali)-[~]
$ dmitry -winse google.com
Deepmagic Information Gathering Tool
"There be some deep magic going on"
```

```
HostIP:142.251.220.14
HostName:google.com
```

```
Gathered Inet-whois information for 142.251.220.14
```

```
-----
inetnum:          142.248.0.0 - 143.13.255.255
ADDRESS-BLOCK
descr:            IPv4 address block not managed by the RIPE NCC
remarks:          -----
remarks:          For registration information,
remarks:          you can consult the following sources:
remarks:          IANA
remarks:          http://www.iana.org/assignments/ipv4-address-space
remarks:          http://www.iana.org/assignments/iana-ipv4-special-registry
-space
remarks:          AFRINIC (Africa)
remarks:          http://www.afrinic.net/ whois.afrinic.net
remarks:          APNIC (Asia Pacific)
remarks:          http://www.apnic.net/ whois.apnic.net
remarks:          ARIN (Northern America)
remarks:          http://www.arin.net/ whois.arin.net
remarks:          LACNIC (Latin America and the Carribean)
remarks:          http://www.lacnic.net/ whois.lacnic.net
remarks:          -----
country:          EU # Country is really world wide
admin-c:          IANA1-RIPE
tech-c:           IANA1-RIPE
status:           ALLOCATED UNSPECIFIED
mnt-by:           RIPE-NCC-HM-MNT
created:          2025-04-16T11:49:16Z
last-modified:    2025-04-16T11:49:16Z
source:           RIPE
```

```
role:          Internet Assigned Numbers Authority
address:       see http://www.iana.org.
admin-c:       IANA1-RIPE
tech-c:        IANA1-RIPE
nic-hdl:       IANA1-RIPE
services
remarks:       go to IANA web site at http://www.iana.org.
mnt-by:        RIPE-NCC-MNT
created:       1970-01-01T00:00:00Z
last-modified: 2001-09-22T09:31:27Z
source:        RIPE # Filtered
```

% This query was served by the RIPE Database Query Service version 1.118.1 (SHETLAND)

Gathered Inic-whois information for google.com

```
-----
PuTTYPuTTYPuTTYPuTTY  Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
d
```

```
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2025-09-02T05:20:19Z <<<
```

For more information on Whois status codes, please visit <https://icann.org/epp>

NOTICE: The expiration date displayed in this record is the date the domain is currently set to expire. This date does not necessarily reflect the expiration date of the domain name registrant's agreement with the sponsoring registrar. Users may consult the sponsoring registrar's Whois database to view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois database through the use of electronic processes that are high-volume and necessary to register domain names or modify existing registrations; the Data in VeriSign Global Registry Services' ("VeriSign") Whois database is provided by VeriSign for information purposes only, and to assist persons in obtaining information about or related to a domain name registration record. VeriSign does not guarantee its accuracy. By submitting a Whois query, you agree to abide by the following terms of use: You agree that you may use this Data only in the following circumstances: (1) allow, enable, or otherwise support the transmission of mass unsolicited, commercial advertising or solicitations via e-mail, telephone, or facsimile; or (2) enable high volume, automated, electronic processes that apply to VeriSign (or its computer systems). The compilation, repackaging, dissemination or other use of this Data is expressly prohibited without the prior written consent of VeriSign. You agree not to use this Data for any other purpose.

You agree that you may use this Data only in the following circumstances: (1) allow, enable, or otherwise support the transmission of mass unsolicited, commercial advertising or solicitations via e-mail, telephone, or facsimile; or (2) enable high volume, automated, electronic processes that apply to VeriSign (or its computer systems). The compilation, repackaging, dissemination or other use of this Data is expressly prohibited without the prior written consent of VeriSign. You agree not to use this Data for any other purpose.

e contains ONLY .COM, .NET, .EDU domains and Registrars.

Gathered Netcraft information for google.com

Retrieving Netcraft.com information for google.com
Netcraft.com Information gathered

Gathered Subdomain information for google.com

Searching Google.com:80...
HostName:support.google.com
HostIP:142.250.67.238
Searching Altavista.com:80...
Found 1 possible subdomain(s) for host google.com, Searched 0 pages containing 0 results

Gathered E-Mail information for google.com

Searching Google.com:80...
Searching Altavista.com:80...
Found 0 E-Mail(s) for host google.com, Searched 0 pages containing 0 results

All scans completed, exiting

Each Line Explanation

```
└──(kali㉿kali)- [~]
```

```
└─$ dmitry -winse google.com
```

dmitry = Deepmagic Information Gathering Tool (used for footprinting / recon)

Flags -winse mean:

- w → Perform whois lookups (IP & domain).
- i → Do a netcraft.com lookup.
- n → Do a netcraft.com DNS lookup.
- s → Search for subdomains.
- e → Search for email addresses.

Target: **google.com**

Banner

Deepmagic Information Gathering Tool

"There be some deep magic going on"

Just the tool's signature line.

Basic Host Info

HostIP:142.251.220.14

HostName:google.com

IP address of google.com resolved as 142.251.220.14.

Hostname confirmed as google.com.

IP WHOIS Info (Inet-Whois)

Gathered Inet-whois information for 142.251.220.14

This section retrieves whois data for the IP address.

inetnum: 142.248.0.0 - 143.13.255.255

This is the range of IPs that the address belongs to.

(142.251.220.14 falls in this block).

descr: IPv4 address block not managed by the RIPE NCC

RIPE (Europe's registry) doesn't directly manage this.

It's a global allocation, handled by IANA (Internet Assigned Numbers Authority).

remarks: ... IANA, AFRINIC, APNIC, ARIN, LACNIC

Explains which registry you should consult depending on region:

AFRINIC → Africa

APNIC → Asia Pacific

ARIN → North America

LACNIC → Latin America & Caribbean

RIPE NCC → Europe

country: EU # Country is really world wide

IP block is global, not tied to a single country.

admin-c / tech-c: IANA1-RIPE

Administrative and technical contact = IANA.

status: ALLOCATED UNSPECIFIED

Block allocated but not tied to a specific organization here.

role: Internet Assigned Numbers Authority

address: see <http://www.iana.org>.

Confirms IANA as manager of this IP block.

created: 2025-04-16T11:49:16Z

last-modified: 2025-04-16T11:49:16Z

Record timestamps.

% This query was served by the RIPE Database Query Service...

This is a footer from the RIPE whois server.

Domain WHOIS Info

Gathered Inic-whois information for google.com

Now it checks domain registration whois.

Domain Name: GOOGLE.COM

Registry Domain ID: 2138514_DOMAIN_COM-VRSN

Unique ID for the domain in the registry.

Registrar WHOIS Server: whois.markmonitor.com

Registrar URL: <http://www.markmonitor.com>

Google uses MarkMonitor registrar.

Updated Date: 2019-09-09

Creation Date: 1997-09-15

Registry Expiry Date: 2028-09-14

Created in 1997, expires 2028.

Registrar Abuse Contact Email: abusecomplaints@markmonitor.com

Contact info for abuse reports.

Domain Status: clientDeleteProhibited

Domain Status: clientTransferProhibited

Domain Status: clientUpdateProhibited

Domain Status: serverDeleteProhibited

Domain Status: serverTransferProhibited

These are ICANN EPP status codes meaning:

Domain cannot be easily deleted, updated, or transferred.

Prevents unauthorized hijacking.

Name Server: NS1.GOOGLE.COM

Name Server: NS4.GOOGLE.COM

Google's authoritative DNS servers.

DNSSEC: unsigned

No DNSSEC security for this domain.

>>> Last update of whois database: 2025-09-02T05:20:19Z <<<

Fresh query timestamp.

♦ Netcraft Info

Gathered Netcraft information for google.com

Retrieving Netcraft.com information for google.com

Netcraft.com Information gathered

Queries Netcraft (a web service for site/server info).

Sometimes it reports hosting provider, OS, uptime — here it shows minimal info.

♦ Subdomain Info

Gathered Subdomain information for google.com

Searching Google.com:80...

HostName: support.google.com

HostIP:142.250.67.238

Found one subdomain: support.google.com (IP 142.250.67.238).

Searching Altavista.com:80...

Found 1 possible subdomain(s) for host google.com

Old method — checks Altavista (historic search engine).

Just confirms 1 subdomain found.

◇ Email Info

Gathered E-Mail information for google.com

Searching Google.com:80...

Searching Altavista.com:80...

Found 0 E-Mail(s) for host google.com

Dmitry also tries to scrape emails from webpages.

Found none.

◇ End of Scan

All scans completed, exiting

Dmitry finished all tasks.

Second example

```
(kali㉿ kali)-[~]
$ dmitry -winse swarnandhra.ac.in
Deepmagic Information Gathering Tool
"There be some deep magic going on"

HostIP:3.110.18.185
HostName:swarnandhra.ac.in

Gathered Inet-whois information for 3.110.18.185
-----

inetnum:        3.0.0.0 - 4.255.255.255
netname:        NON-RIPE-NCC-MANAGED-ADDRESS-BLOCK
descr:          IPv4 address block not managed by the RIPE NCC
remarks:        -----
remarks:        For registration information,
remarks:        you can consult the following sources:
remarks:        IANA
remarks:        http://www.iana.org/assignments/ipv4-address-space
remarks:        http://www.iana.org/assignments/iana-ipv4-special-registry
remarks:        http://www.iana.org/assignments/ipv4-recovered-address-space
remarks:
```

```

remarks:      AFRINIC (Africa)
remarks:      http://www.afrinic.net/ whois.afrinic.net
remarks:
remarks:      APNIC (Asia Pacific)
remarks:      http://www.apnic.net/ whois.apnic.net
remarks:
remarks:      ARIN (Northern America)
remarks:      http://www.arin.net/ whois.arin.net
remarks:
arribean)
remarks:      http://www.lacnic.net/ whois.lacnic.net
remarks:
remarks:      -----
country:      EU # Country is really world wide
admin-c:      IANA1-RIPE
tech-c:       IANA1-RIPE
status:       ALLOCATED UNSPECIFIED
mnt-by:       RIPE-NCC-HM-MNT
created:      2019-01-07T10:50:00Z
last-modified: 2019-01-07T10:50:00Z
source:       RIPE

role:         Internet Assigned Numbers Authority
ana.org.
admin-c:      IANA1-RIPE
tech-c:       IANA1-RIPE
nic-hdl:      IANA1-RIPE
remarks:      For more information on IANA services
remarks:      go to IANA web site at http://www.iana.org.
mnt-by:       RIPE-NCC-MNT
created:      1970-01-01T00:00:00Z
last-modified: 2001-09-22T09:31:27Z
source:       RIPE # Filtered
% This query was served by the RIPE Database Query Service version 1.118.1 (SHET
LAND)

```

```

Gathered Inic-whois information for swarnandhra.ac.in
-----
PuTTYPuTTYPuTTYPuTTYError: Unable to connect - Invalid Host
ERROR: Connection to InicWhois Server in.whois-servers.net failed
close error

Gathered Netcraft information for swarnandhra.ac.in
-----

Retrieving Netcraft.com information for swarnandhra.ac.in
Netcraft.com Information gathered

Gathered Subdomain information for swarnandhra.ac.in
-----
Searching Google.com:80...
Searching Altavista.com:80...
Found 0 possible subdomain(s) for host swarnandhra.ac.in, Searched 0 pages conta
ining 0 results

Gathered E-Mail information for swarnandhra.ac.in
-----
Searching Google.com:80...
Searching Altavista.com:80...
Found 0 E-Mail(s) for host swarnandhra.ac.in, Searched 0 pages containing 0 resu
lts

All scans completed, exiting

```

You ran

`dmitry -winse swarnandhra.ac.in`

Let's break this scan result **line by line in detail** so you clearly understand what happened.

Command Recap

- -w → Whois lookup (IP & domain)
- -i → Netcraft lookup
- -n → Netcraft DNS lookup
- -s → Subdomain search
- -e → Email search

◇ Basic Info

HostIP:3.110.18.185

HostName:swarnandhra.ac.in

- swarnandhra.ac.in resolves to IP **3.110.18.185**.
- That IP is part of **Amazon AWS (ap-south-1 region, Mumbai, India)**.
- Hostname confirmed as the same domain.

◇ IP Whois (Inet-Whois)

inetnum: 3.0.0.0 - 4.255.255.255

netname: NON-RIPE-NCC-MANAGED-ADDRESS-BLOCK

descr: IPv4 address block not managed by the RIPE NCC

- This means the IP range belongs to **ARIN (North America)**, not RIPE (Europe).
- RIPE is just relaying info, saying: *"Check ARIN/IANA for details."*

remarks: For registration information, you can consult IANA, AFRINIC, APNIC, ARIN, LACNIC

- Points you to the correct RIRs (Regional Internet Registries).

country: EU # Country is really world wide

- Default placeholder (since AWS is global). Actual location = India (Mumbai).

created: 2019-01-07

last-modified: 2019-01-07

- This IP range allocation was recorded in **2019**.

role: Internet Assigned Numbers Authority

address: see <http://www.iana.org>.

- Confirms IANA as coordinator of global IP address distribution.

Meaning:

The IP **3.110.18.185** is owned by **Amazon AWS**, used in India region, but the whois shown is the generic RIPE/IANA record. For exact owner, you'd query **ARIN or APNIC**.

♦ Domain Whois (Inic-Whois)

Error: Unable to connect - Invalid Host

ERROR: Connection to InicWhois Server in.whois-servers.net failed

- Dmitry tried to query the **.in registry WHOIS server** (in.whois-servers.net) but failed.
- Possible reasons:
 1. The server blocked the request.
 2. Dmitry's method is outdated for .in domains.
 3. The WHOIS server requires manual web lookup.

So **no domain registration info** (like registrar, owner, expiry) was retrieved.

♦ Netcraft Info

Retrieving Netcraft.com information for swarnandhra.ac.in

Netcraft.com Information gathered

- Dmitry pinged Netcraft, but output is minimal here.
- Normally this could show:
 - Hosting: Amazon AWS
 - Server: Apache/Nginx
 - SSL details

◇ Subdomain Info

Searching Google.com:80...

Searching Altavista.com:80...

Found 0 possible subdomain(s) for host swarnandhra.ac.in

- Dmitry tried scraping **Google & Altavista** search results.
- Found **0 subdomains**.
- (This method is very old — tools like **Sublist3r** / **Amass** work better).

◇ Email Info

Searching Google.com:80...

Searching Altavista.com:80...

Found 0 E-Mail(s) for host swarnandhra.ac.in

- Searched for **email addresses** linked to the domain.
- None found.

◇ End

All scans completed, exiting

- Dmitry finished its tasks.

☑ Final Summary (for swarnandhra.ac.in)

- **Domain:** swarnandhra.ac.in
- **IP:** 3.110.18.185 → Amazon AWS (Mumbai, India)
- **IP Whois:** Generic RIPE/IANA record (real details in ARIN/APNIC)
- **Domain Whois:** Failed (needs manual lookup for .in)
- **Netcraft Info:** Minimal (likely AWS + Apache/Nginx)
- **Subdomains:** None detected
- **Emails:** None detected

You noticed that in your **first test (google.com)** Dmitry gave you **lots of info** (registrar, creation date, expiry date, name servers, subdomains etc.), but in your **second test (swarnandhra.ac.in)** it showed **less info** (domain WHOIS failed, no subdomains, no emails).

Let's carefully explain **why this difference happens**

♦ 1. Domain Extensions (.com vs .in)

- google.com = belongs to the **.com TLD** (top-level domain), managed by **Verisign**.
 - Verisign has **open WHOIS servers** that Dmitry can query easily.
 - That's why you saw registrar details, creation/expiry dates, and domain statuses.
- swarnandhra.ac.in = belongs to the **.in TLD** (India).
 - .in domains are managed by **NIXI (National Internet Exchange of India)**.
 - The **WHOIS server (in.whois-servers.net)** often blocks automated tools like Dmitry.
 - That's why Dmitry gave:
 - Error: Unable to connect - Invalid Host
 - ERROR: Connection to InicWhois Server in.whois-servers.net failed
 - ☒ If you manually check using:
 - whois swarnandhra.ac.in

You will see registrar details, but Dmitry can't fetch it automatically.

♦ 2. Hosting Type (AWS vs Google)

- **Google.com** → Has **lots of public records**, Netcraft data, thousands of subdomains, etc.
- **Swarnandhra.ac.in** → Hosted on **AWS (Amazon Cloud)**.
 - AWS IP ranges are huge and shared.
 - The Whois only shows "Amazon AWS" (not the college directly).
 - That's why you only got a **generic IANA/RIPE record**, instead of detailed organization info.

◇ 3. Subdomain & Email Search

- Dmitry uses **old scraping methods**:
 - Google.com:80
 - Altavista.com:80 (Altavista is dead since 2013).
- For **Google.com** → Popular domain, so even outdated scraping found at least support.google.com.
- For **Swarnandhra.ac.in** → Much smaller site, not many indexed subdomains or emails.
 - So Dmitry reported **0 results**.

Modern tools like **Sublist3r**, **Amass**, **theHarvester** give much better results.

◇ 4. Netcraft Info

- Netcraft has **plenty of info for Google.com** (world's most visited site).
- For smaller sites (like swarnandhra.ac.in), Netcraft may not have much recorded.
- That's why it just said:
- Netcraft.com Information gathered

but gave no details.

☑ Final Explanation (Why Less Info Came)

1. **.in registry** blocks automated WHOIS queries → Dmitry couldn't fetch registrar info.
2. **AWS hosting** → Whois shows Amazon's block, not the college directly.
3. **Few subdomains/emails indexed** → Dmitry's old scraping methods found nothing.
4. **Netcraft coverage** → Smaller sites often have little or no public data there.

EXP-6: Work with Sniffers for Monitoring Network Communication (Wireshark)

Prerequisites:

- Basic understanding of TCP/IP, Ethernet, and network protocols (HTTP, DNS, TCP, UDP, ARP).
- Familiarity with command-line basics.
- Wireshark installed on the lab machines (latest stable release recommended).

Learning Objectives

1. Understand how packet sniffers capture and display network traffic.
2. Use Wireshark to capture live traffic on an interface.
3. Apply capture and display filters to isolate relevant packets.
4. Inspect packet contents: Ethernet, IP, TCP/UDP headers and application payloads.
5. Reconstruct TCP streams and HTTP sessions.
6. Use Wireshark statistics and tools to summarize traffic and identify anomalies.
7. Export and save capture files (.pcapng) and use tshark for command-line analysis.

Safety & Legal Notice

- Only capture traffic on networks and devices for which you have explicit permission. Capturing someone else's private traffic without consent may be illegal and unethical.
- In a lab environment, use isolated networks, virtual machines, or intentionally generated traffic.

Equipment & Software

- Desktop or laptop with Wireshark installed.
- Network connection (wired preferred for stable captures) or isolated virtual network.
- Optional: Two VMs (one as client, one as server) to generate controlled traffic.

Lab Setup Instructions

1. Ensure Wireshark is installed. Launch Wireshark with sufficient privileges to capture from network interfaces (on Linux use sudo or run as root; on Windows run as Administrator).
2. Identify the network interface to capture from (e.g., Ethernet, Wi-Fi, vboxnet0, vmnet1).
3. If using virtual machines, set their network adapter to "Host-only" or "NAT with port forwarding" depending on the exercise.

Exercise 1 — Basic Capture and Filters

Goal: Capture live traffic and use display filters to inspect packets.

Steps:

1. Open Wireshark and start capturing on the chosen interface.
2. Generate network activity from the host (open a web page, ping a remote host, perform DNS lookups).
3. Stop capture after ~30–60 seconds.

Tasks:

- Use display filters to find specific traffic types (try these in the filter bar):
 - `icmp` — show ICMP (ping) packets.
 - `http` — show HTTP requests/responses.
 - `tcp.port == 80` — show TCP traffic on port 80.
 - `ip.addr == 192.168.1.10` — show packets to/from an IP address.
- Save the capture as `lab6_ex1_<yourname>.pcapng`.

Questions:

1. How many ICMP echo requests and replies were captured?
2. Which IP addresses were involved in your web browsing activity?

Exercise 2 — Capture Filters vs Display Filters

Goal: Learn the difference and apply capture filters to limit data.

Notes:

- Capture filters (set before starting capture) use libpcap syntax and limit what data is recorded.
- Display filters (applied after capture) filter what you see without changing the recorded file.

Steps:

1. In Wireshark, click the capture interface settings and set the capture filter to port 53 or port 80.
2. Start capture and perform a web browse and a DNS lookup (e.g., `nslookup www.example.com`).
3. Stop capture.

Tasks:

- Compare the file size and packet count to the full capture from Exercise 1.
- Try capture filter examples:
 - host 192.168.1.5 — capture only traffic to/from a host.
 - tcp and not port 22 — capture TCP traffic excluding SSH.

Questions:

1. What are the advantages and disadvantages of using capture filters?

Exercise 3 — Protocol Dissection and Field Analysis

Goal: Inspect common protocol fields and headers.

Steps:

1. Open a capture file (use your saved file or a provided sample .pcapng).
2. Select a TCP packet containing an HTTP request.
3. Expand the packet details pane to view:
 - Frame
 - Ethernet II (src/dst MAC)
 - Internet Protocol Version 4 (src/dst IP, TTL)
 - Transmission Control Protocol (src/dst ports, seq/ack numbers, flags)
 - Hypertext Transfer Protocol (request/response headers)

Tasks:

- Identify the TCP flags set in the SYN, SYN/ACK, and FIN packets.
- Note the HTTP Host and User-Agent headers for a captured request.

Questions:

1. How does the sequence and acknowledgment numbering work during the TCP handshake?

Exercise 4 — Follow TCP Stream & Reassemble

Goal: Reconstruct conversations and view payloads as readable text.

Steps:

1. Right-click an HTTP packet and select **Follow > TCP Stream**.
2. Observe the full request/response conversation in text.
3. Use the menu to switch view from ASCII to EBCDIC or raw if needed.

Tasks:

- Save the reconstructed stream to a file.
- Reconstruct an SMTP or FTP conversation (if available) and identify credentials or commands (only in lab-controlled environment).

Questions:

1. Which HTTP status codes did you observe? What do they indicate?

Exercise 5 — DNS and ARP Analysis

Goal: Inspect name resolution and layer-2 address translation.

Steps:

1. Filter for DNS traffic: `dns or udp.port == 53`.
2. Perform DNS lookups from the command line and watch the traffic.
3. Filter for ARP: `arp` and observe ARP requests and replies.

Tasks:

- Identify query types (A, AAAA, PTR, CNAME).
- For ARP, note the sender and target hardware/IP addresses.

Questions:

1. What is the difference between iterative and recursive DNS queries (as observed in packets)?

Exercise 6 — TLS/HTTPS Analysis (What You Can See)

Goal: Understand limits when traffic is encrypted and what metadata remains visible.

Steps:

1. Capture an HTTPS session (browse to <https://www.example.com>).
2. Use display filters: `tls` or `ssl`.

Observations:

- You will see TCP handshake and TLS handshake messages but not application payload.
- You can observe server certificates (ClientHello/ServerHello, certificate chain), SNI (Server Name Indication) if not encrypted by TLS 1.3 ESNI, and cipher suites.

Tasks:

- Inspect the TLS Client Hello and list the cipher suites offered.
- View the server certificate details (subject, issuer, validity period).

Questions:

1. Why can't Wireshark display HTTPS payload by default? How can you decrypt TLS traffic in Wireshark (lab-only, with keys)?

Exercise 7 — Using Wireshark Statistics and Tools

Goal: Use built-in statistics to summarize traffic and identify anomalies.

Steps & Tasks:

- **Statistics > Protocol Hierarchy:** Observe protocol breakdown and byte/packet counts.
- **Statistics > Conversations:** View top talkers (IP/IP, TCP, UDP).
- **Statistics > Endpoints:** See which endpoints exchanged the most traffic.
- **Statistics > IO Graphs:** Plot packet/byte rates over time; create graphs per protocol.
- **Analyze > Expert Information:** Review potential problems, warnings, malformed packets.

Questions:

1. Which IP address generated the most traffic in your capture?
2. Did the Expert Information flag anything suspicious?

Exercise 8 — Exporting, Scripting, and tshark

Goal: Save captures, export data, and perform command-line analysis.

Steps:

1. File > Export Specified Packets or File > Export Packet Dissections to CSV/PDML/TEXT.
2. Save the capture as exp6_full_<yourname>.pcapng.
3. Use tshark (Wireshark's CLI):
 - List interfaces: tshark -D
 - Capture to file: tshark -i 1 -w capture.pcapng -a duration:30
 - Read and filter: tshark -r capture.pcapng -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri

Tasks:

- Export all HTTP request URIs seen to a text file using tshark.

Challenge Task — Detecting a Simple ARP Spoofing Attempt (Controlled Lab)

Goal: Use Wireshark to identify signs of ARP spoofing.

Setup (lab-only, controlled):

- Configure a VM to run an ARP spoofing tool against another lab VM.
- Capture traffic between VMs.

What to look for:

- Multiple ARP Replies claiming the same IP but with different MAC addresses.
- Duplicate IP addresses seen in ARP responses.
- An unusual increase in ARP traffic.

Questions:

1. Which packets indicate possible ARP poisoning?
2. How would you mitigate ARP spoofing on a real network?

Lab Report Requirements

Submit a short report (1–2 pages) including:

1. Objectives and setup description.
2. Steps performed and filters used.
3. Key observations and screenshots (Wireshark window with important packets highlighted).
4. Answers to questions in each exercise.